

30-Day AI/ML Roadmap — Personalized for a CS Student

Your profile: Student, comfortable with Python & its libraries, some ML and prob/calc exposure already, frontend background, ~3 hrs/day, learns best through videos + hands-on projects.

How this plan uses your background: - No time wasted on “what is a variable” or “what is a derivative” — you’ll get fast refreshers, not full lectures. - Your frontend skills get used in Week 4 to turn a model into something visual and shareable (most ML learners skip this and it’s a huge portfolio differentiator). - Every day pairs a short video with immediate hands-on code — no passive watching without doing.

Daily time budget (3 hrs): ~45–60 min video/reading, ~90–120 min coding.

Week 1: Refresh + Core ML Workflow

Milestone: Skip straight past basics you already know; get a working, evaluated ML model by Day 7 using your existing Python skills.

Day	Task (3 hrs)	Resources
1	Refresh NumPy/Pandas only where rusty (skim, don’t rewatch full courses). Load a real dataset, do quick EDA.	Pandas Cheat Sheet (video, 20 min) ; Kaggle Titanic
2	Prob/stats refresher <i>specifically for ML</i> : distributions, expectation, variance, Bayes’ theorem — the parts ML actually uses.	StatQuest: “Bayes Theorem” + “Probability vs Likelihood” (videos)
3	Train/test split, overfitting/underfitting, bias-variance tradeoff — watch, then implement on your Day 1 dataset.	StatQuest: “Bias and Variance” (video)
4	Linear & logistic regression: watch the StatQuest pair, then implement both from scratch in NumPy (no sklearn) to actually feel the math.	StatQuest: “Linear Regression” + “Logistic Regression”
5	Re-implement Day 4 models using scikit-learn; compare to your from-scratch versions.	scikit-learn linear_model docs
6	Model evaluation metrics: precision/recall/F1/ROC-AUC. Apply all of them to your Day 5 model.	StatQuest: “Confusion Matrix” + “ROC and AUC” (videos)
7	Project Day. Full pipeline end-to-end on a dataset of your choice (not Titanic — pick something you’re curious about).	Kaggle Datasets

Week 1 Project: A from-scratch + scikit-learn comparison classifier/regressor on a dataset you picked yourself, with a short markdown writeup of what changed between your manual implementation and sklearn’s.

Week 2: Trees, Ensembles & Real Feature Engineering

Milestone: Be able to confidently beat a baseline model using ensembles and good feature engineering — the actual day-to-day skill of applied ML.

Day	Task (3 hrs)	Resources
-----	--------------	-----------

8	Decision trees (watch) → implement with sklearn → visualize the tree.	StatQuest: “Decision Trees” (video)
9	Random Forests & bagging — why ensembles reduce variance. Apply to Week 1’s dataset, compare scores.	StatQuest: “Random Forest” (video)
10	Gradient Boosting (XGBoost) — watch, then swap into your pipeline.	StatQuest: “Gradient Boost” (video); XGBoost docs
11	Feature engineering deep-dive: encoding, scaling, date features, interaction terms — the stuff that beats fancier models.	Kaggle Feature Engineering micro-course (free)
12	Cross-validation + hyperparameter tuning (GridSearchCV/Optuna).	Optuna quickstart (video walkthrough)
13	Enter a live/recent Kaggle tabular competition (or a “Getting Started” one) — do EDA + feature pass.	Kaggle Competitions
14	Project Day. Push your competition entry: ensemble + tuned hyperparameters + feature engineering, document score improvement at each step.	—

Week 2 Project: A Kaggle competition submission with a “model improvement log” — baseline score → feature engineering → ensemble → tuned, with the score at each step (great interview talking point).

Week 3: Deep Learning — Neural Nets & CNNs

Milestone: Go from “I know ML” to “I can build and train neural networks,” including image models, using PyTorch.

Day	Task (3 hrs)	Resources
15	Neural net fundamentals — given your calc background, lean into the math, don’t skip it.	3Blue1Brown: “Neural Networks” (videos, ep 1-2)
16	Backpropagation, properly understood (your calc helps here a lot).	3Blue1Brown: “Backpropagation” + “Backprop calculus” (videos)
17	PyTorch basics: tensors, autograd, build a feedforward net from scratch.	PyTorch 60-min Blitz (interactive, do every cell)
18	Train your feedforward net on MNIST; plot loss curves; experiment with learning rate.	PyTorch MNIST tutorial
19	CNNs: convolution & pooling intuition.	CNN Explainer (interactive visual tool) — great for visual learners
20	Train a CNN on CIFAR-10 from scratch.	PyTorch CIFAR-10 tutorial
21	Project Day. Transfer learning: fine-tune a pretrained ResNet on a dataset you choose; compare vs. your from-scratch CNN.	PyTorch Transfer Learning tutorial

Week 3 Project: An image classifier on a topic you pick (not generic cats/dogs — something you’d actually use), comparing from-scratch CNN vs. transfer learning on accuracy and training time.

Week 4: Transformers + Capstone (with Your Frontend Edge)

Milestone: Understand modern NLP/LLM foundations, and — using your frontend skills — ship a deployed, demoable capstone project instead of just a notebook.

Day	Task (3 hrs)	Resources
22	Attention & Transformers conceptually — this is “how ChatGPT-like models work” day.	Jay Alammar: “The Illustrated Transformer” + 3Blue1Brown “Attention in transformers” (video)
23	Hugging Face transformers : load pretrained models, run inference tasks (sentiment, summarization, NER).	HF Quicktour
24	Fine-tune a small pretrained transformer on a custom text dataset.	HF Fine-tuning tutorial
25	Embeddings + vector search basics (semantic search) — foundation for RAG apps.	Sentence-Transformers docs
26	Build a simple FastAPI/Flask backend that serves your Week 3 or Week 4 model via an API endpoint.	FastAPI tutorial
27	Capstone build: connect your existing frontend skills to the API — build a small UI (React/HTML+JS) that calls your model and shows results live.	Reuse your frontend stack
28	Capstone polish: error handling, loading states, a clean UI pass.	—
29	Deploy it (Streamlit Cloud / Render / Vercel for frontend + a free-tier backend host) so it’s a live link, not just code.	Streamlit Cloud docs or Render docs
30	Wrap-up. Write a clean README + short writeup of what you built and learned; post it (GitHub + LinkedIn/portfolio).	—

Capstone idea (uses your full stack): A deployed web app where a user uploads an image or types text, your fine-tuned model (Week 3 or 4) processes it via a FastAPI backend, and your own frontend displays the result live — e.g. an image classifier with a drag-and-drop UI, or a text sentiment/summarizer tool.

Resource Summary

- **Video-first explainers:** StatQuest (classical ML/stats), 3Blue1Brown (neural net math), [CNN Explainer](#) (interactive visual)
- **Hands-on docs:** [scikit-learn](#), [PyTorch tutorials](#), [Hugging Face docs](#)
- **Practice/competitions:** [Kaggle](#), [Kaggle Learn micro-courses](#) (free, fast, hands-on)
- **Deployment (your frontend edge):** [FastAPI](#), [Streamlit Cloud](#), [Render](#)
- **Compute:** [Google Colab](#) (free GPU)

Final Outcome (End of Day 30)

By Day 30, you will have:

1. **4 real projects on GitHub**, including one **fully deployed, live demo** — not just a notebook — combining your

ML skills with your existing frontend skills.

2. **A “model improvement log”** from Week 2 that’s a genuinely strong interview talking point (shows you understand *why* a model improves, not just that it does).
3. **From-scratch implementations** of linear/logistic regression and a CNN, so you actually understand the math, not just the `.fit()` call.
4. **Practical fluency** in the real ML stack: Pandas, scikit-learn, PyTorch, Hugging Face, plus a basic deployment pipeline (FastAPI + hosting).
5. **A clear next direction** — whether that’s going deeper into deep learning/research, MLOps, or specializing in CV/NLP — based on which week you enjoyed most.

Pacing note: If a topic doesn’t click in one day (backprop and attention are the usual culprits), spend an extra day on it and compress a lighter day. The weekly projects, not the daily schedule, are your real checkpoints.